

■■ Chama Management & Payment Platform

A production-oriented Chama management, financial management, payment and reconciliation platform designed for Kenyan Chamas.

Provider-agnostic architecture integrating Jenga, KCB BUNI, NCBA and Safaricom Daraja.

ChamaCore

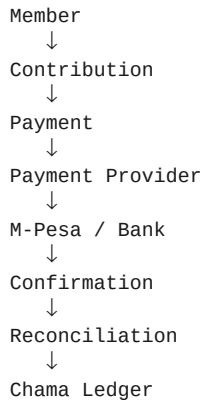
Technical Specification & Architecture Guide

The Chama business logic must never depend directly on a specific bank or payment provider.

1. Project Vision

Traditional Chamas often manage member registration, monthly contributions, shares, loans, loan repayments, payouts, bank deposits, M-Pesa payments and financial records using books, spreadsheets, WhatsApp messages and manual reconciliation.

This project aims to provide one system where the full financial pipeline is automated:

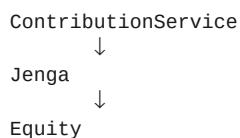


The goal is not simply to build CRUD screens. The goal is to build a real financial system with proper transaction tracking, reconciliation, auditability and provider integrations.

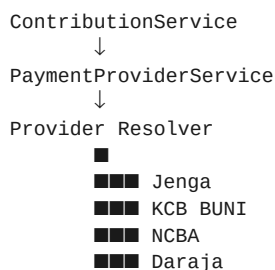
2. Core Principle

The Chama system must not depend on one payment provider.

Bad architecture:



Better architecture:



This allows a Chama using Equity to use Jenga while another Chama using KCB can use KCB BUNI. The business layer does not need to change.

3. Technology Stack

Backend

- Python
- FastAPI
- PostgreSQL
- SQLAlchemy
- Alembic

- Pydantic
- JWT authentication
- REST API

External integrations

- Jenga API
- KCB BUNI
- NCBA APIs / collection services
- Safaricom Daraja

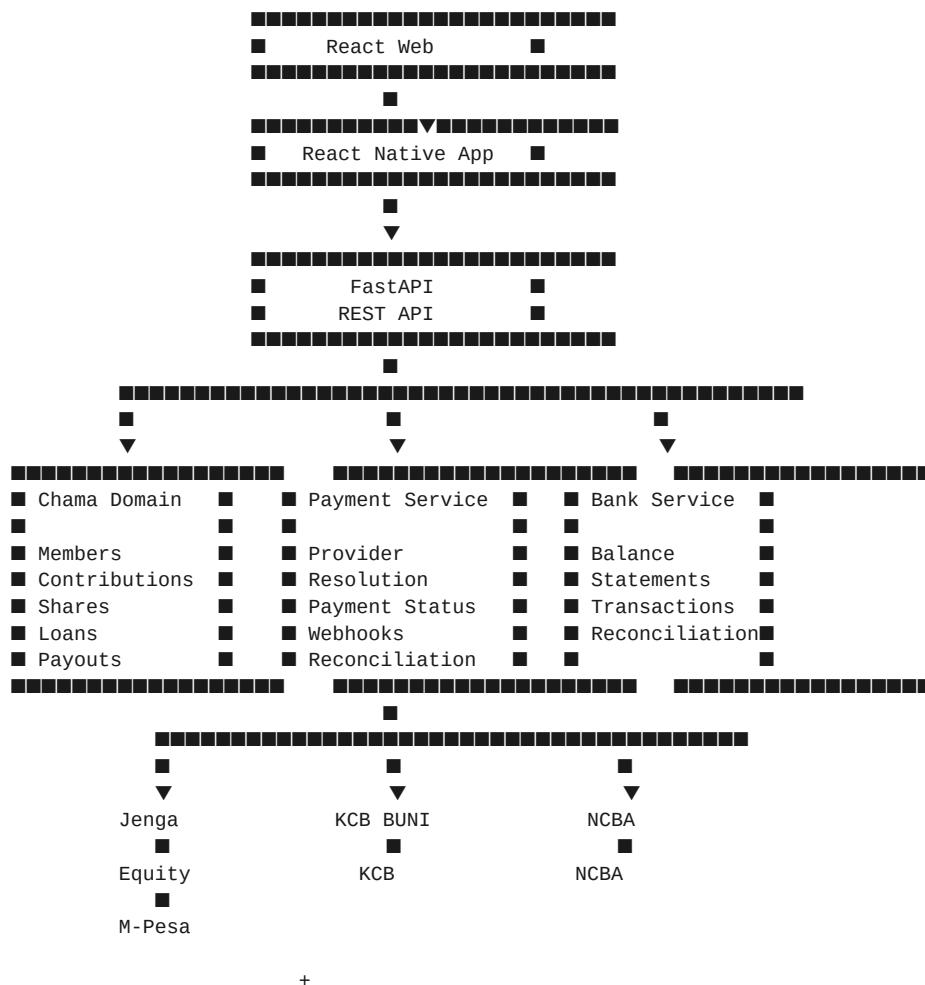
Frontend (planned)

- React
- React Native

Future

- USSD
- SMS
- Background workers
- Notifications
- Reporting
- Monitoring

4. High-Level Architecture



5. Main Modules

The system is divided into the following domains (implemented incrementally):

1. Authentication
2. Chama Management
3. Member Management
4. Contributions
5. Shares
6. Loans
7. Loan Repayments
8. Payouts
9. Ledger
10. Payments
11. Payment Providers
12. Bank Accounts
13. Reconciliation
14. Notifications
15. Reports
16. Audit
17. USSD

6. Development Roadmap — V1 Chama Foundation

Build the basic Chama management system.

Chama

- Create / Update / View / Delete or deactivate Chama
- Chama profile
- Contribution rules

Members

- Register / Add / Update / Remove or deactivate member
- View member
- Member history

Roles

CHAIRPERSON · TREASURER · SECRETARY · MEMBER

Registration Fee

Track: Member · Registration Fee · Payment Status · Date

Contributions (manual initially)

Member
↓
Contribution
↓
Amount

↓
Month

No payment provider integration yet.

7. V2 — Financial Management

Contributions

Contribution → Member · Chama · Amount · Period · Status · Date

Shares

Member → Shares → Share Value

Loans

Loan → Member · Principal · Interest · Total Due · Status · Due Date

Loan Repayments

Loan → Loan Repayment → Payment

Payouts

Track money leaving the Chama.

8. Financial Ledger

The application must not rely on a manually modified balance. Financial activity must be represented by transactions. The ledger becomes the financial source of truth.

John Contribution	+1,000
Mary Contribution	+1,000
Loan to John	-5,000
Loan repayment	+2,000

Current balance	-1,000

9. V3 — Payment Architecture

Introduce a provider-independent payment system.

Payment entities

- Payment
- PaymentAttempt
- ProviderTransaction
- ProviderCallback
- PaymentProviderConfiguration

Payment states

PENDING · PROCESSING · COMPLETED · FAILED · CANCELLED · REVERSED · UNKNOWN

10. PaymentProvider Interface

The application should define an internal interface:

PaymentProvider

■
■■■ initiate_payment()
■■■ get_payment_status()
■■■ handle_callback()
■■■ verify_transaction()
■■■ refund_payment()
■■■ reconcile()

Provider implementations: JengaPaymentProvider · KCBPaymentProvider · NCBAPaymentProvider · DarajaPaymentProvider

The rest of the application communicates only with the interface.

11. BankProvider Interface

Payments and bank-account services are separate concepts.

BankProvider

■
■■■ get_balance()
■■■ get_transactions()
■■■ get_statement()
■■■ validate_account()
■■■ reconcile()

Implementations: JengaBankProvider · KCBBankProvider · NCBABankProvider

12. Provider Coverage

Provider	Primary purpose
Jenga	Payment processing + supported bank services
KCB BUNI	Payments, interoperability and KCB services
NCBA	Collections, payment notifications and banking integrations
Daraja	Direct M-Pesa integration

Provider capability is not assumed to be universal. Provider selection is based on the specific capability required by the Chama, not simply the bank name.

13. V4 — Jenga Integration

The first major payment integration will be Jenga. For an Equity Chama:

Member
↓
FastAPI
↓
PaymentProviderService
↓
JengaPaymentProvider
↓
Jenga API
↓
M-Pesa STK Push
↓
Member enters PIN
↓
Payment
↓
Jenga Callback

↓
FastAPI
↓
Payment Completed
↓
Contribution Paid
↓
Ledger Updated

Jenga supports M-Pesa STK/USSD Push and documents account-based settlement.

14. V5 — Webhooks

Every provider gets its own webhook endpoint:

POST /webhooks/jenga · POST /webhooks/kcb · POST /webhooks/ncba · POST /webhooks/daraja

Provider-specific payloads are converted into one internal format:

```
{  
  "provider": "jenga",  
  "provider_transaction_id": "ABC123",  
  "payment_reference": "P7K92A",  
  "amount": 1000,  
  "currency": "KES",  
  "status": "COMPLETED"  
}
```

Flow: Webhook → Provider Adapter → Normalized Event → Payment Service → Reconciliation → Ledger

15. Idempotency

Financial callbacks can potentially be delivered more than once. The system must never credit a payment twice. Database constraints should include UNIQUE(provider, provider_transaction_id).

Callback #1 ABC123 → Process
Callback #2 ABC123 → Already processed → Ignore

16. Payment References

Every payment receives an internal reference (example: P7K92A).

P7K92A
↓
Payment UUID
↓
Contribution
↓
Member
↓
Chama

The external provider reference and internal database ID must remain separate.

17. V6 — KCB BUNI

Implement KCBPaymentProvider and KCBBankProvider.

```
providers/  
■ kcb/  
  ■ client.py  
  ■ auth.py  
  ■ payments.py  
  ■ accounts.py  
  ■ callbacks.py
```

```
■■■ mapper.py
```

The KCB integration must only expose capabilities actually available to the application's KCB account and approved BUNI products.

18. V7 — NCBA

Implement NCBAPaymentProvider and NCBABankProvider.

```
providers/  
■■■ ncba/  
    ■■■ client.py  
    ■■■ auth.py  
    ■■■ payments.py  
    ■■■ collections.py  
    ■■■ accounts.py  
    ■■■ callbacks.py  
    ■■■ mapper.py
```

NCBA's collection/API products are particularly useful for automatic payment notification and reconciliation.

19. V8 — Daraja

Implement a direct M-Pesa provider (DarajaPaymentProvider).

```
providers/  
■■■ daraja/  
    ■■■ client.py  
    ■■■ auth.py  
    ■■■ stk.py  
    ■■■ c2b.py  
    ■■■ callbacks.py  
    ■■■ mapper.py
```

This gives the platform another option when a Chama wants direct Safaricom M-Pesa integration rather than a bank/provider intermediary.

20. Provider Resolution

When a member makes a payment (POST /api/v1/contributions/{id}/pay), FastAPI determines:

```
Which Chama?  
  ↓  
Which bank?  
  ↓  
Which provider?  
  ↓  
Which payment method?
```

Example:
NUCSWG → Equity → Jenga → M-Pesa STK
XYZ Chama → KCB → KCB BUNI → Supported payment method

21. V9 — Reconciliation Engine

Reconciliation connects external transactions to internal financial records. Never match payments only using amount. Use: provider · provider_transaction_id · payment_reference · amount · timestamp · status.

```
Internal: Contribution #543  John  KSh 1,000  
External: Provider Transaction ABC123  KSh 1,000  COMPLETED
```

```
Reconciliation:  
ABC123 → Payment #900 → Contribution #543 → John → Ledger
```


22. V10 — Bank Reconciliation

Payment confirmation and bank confirmation are separate. Where supported, the system can compare Provider transactions vs Bank transactions vs Chama ledger.

23. V11 — Reports

Member statement: Contributions · Shares · Loans · Repayments · Balance

Chama statement: Opening Balance + Contributions + Loan Repayments – Loans – Payouts = Closing Balance

Financial reports:

- Monthly contributions
- Member contributions
- Loan report
- Loan repayment report
- Payout report
- Provider transaction report
- Bank reconciliation report
- Outstanding contributions
- Outstanding loans

24. V12 — Audit System

Every sensitive financial operation should produce an audit event storing: user · action · entity · entity_id · old_value · new_value · timestamp · IP/device information where appropriate.

Examples: MEMBER_CREATED · CONTRIBUTION_CREATED · PAYMENT_INITIATED · PAYMENT_COMPLETED · LOAN_APPROVED · LOAN_REPAID · PAYOUT_CREATED · BANK TRANSACTION RECONCILED

25. V13 — Notifications

SMS · Email · Push Notifications (asynchronous).

- Contribution received
- Payment failed
- Loan approved
- Loan repayment due
- Monthly contribution reminder
- Payout completed

26. V14 — React Web Application

Primarily for executives. Main sections: [Dashboard](#) · [Members](#) · [Contributions](#) · [Shares](#) · [Loans](#) · [Payouts](#) · [Payments](#) · [Bank Account](#) · [Reconciliation](#) · [Reports](#) · [Audit](#) · [Settings](#).

██		
■ Chama Dashboard		■
██		
■ Members	32	■
■ Contributions	45,000	■
■ Loans	18,000	■
■ Balance	127,500	■

27. V15 — React Native Application

Primarily for members: [View contributions](#) · [Make contribution](#) · [View shares](#) · [Request loan](#) · [View loan](#) · [Make repayment](#) · [View statements](#) · [Receive notifications](#). Communicates with the same FastAPI backend.

28. V16 — USSD

```
*XXXX#
↓
1. My Account
2. Contribution
3. Loan
4. Balance
5. Statement
```

Business logic remains shared across React, React Native and USSD.

29. Backend Project Structure

```

app/
├── api/
│   ├── auth/
│   ├── chamas/
│   ├── members/
│   ├── contributions/
│   ├── shares/
│   ├── loans/
│   ├── payouts/
│   ├── payments/
│   ├── reports/
│   └── webhooks/
├── domain/
│   ├── chamas/
│   ├── members/
│   ├── contributions/
│   ├── shares/
│   ├── loans/
│   ├── payments/
│   └── ledger/
├── services/
│   ├── chama_service.py
│   ├── member_service.py
│   ├── contribution_service.py
│   ├── share_service.py
│   ├── loan_service.py
│   ├── ledger_service.py
│   ├── payment_service.py
│   └── reconciliation_service.py
├── providers/
│   ├── base/
│   │   ├── payment_provider.py
│   │   └── bank_provider.py
│   ├── jenga/
│   ├── kcb/
│   ├── ncba/
│   └── daraja/
├── models/
├── schemas/
├── repositories/
├── db/
├── security/
├── workers/
├── config/
└── main.py

```

30. Database Structure

Core tables

users · chamas · memberships · roles · contributions · shares · loans · loan_repayments · payments · payment_attempts · provider_transactions · provider_callbacks · bank_accounts · bank_transactions · ledger_accounts · ledger_entries · reconciliation_records · audit_logs · notifications

Important relationships

Member → Contribution → Payment → ProviderTransaction → LedgerEntry

Bank side:

BankAccount → BankTransaction → ReconciliationRecord → LedgerEntry

31. Security

-
- Authentication (JWT/session)
 - Password hashing
 - Role-based access control
 - Provider credential encryption
 - HTTPS
 - Input validation
 - Rate limiting
 - Webhook verification
 - Idempotency
 - Audit logs
 - Database backups
 - Secrets management
 - Secure environment variables

Never store API secrets, consumer secrets, passwords, M-Pesa credentials or bank credentials directly in source code.

32. Financial Rules

Financial operations must be treated differently from ordinary CRUD. Do not allow DELETE payment after a successful financial transaction. Instead use REVERSAL or REFUND where supported. Historical ledger entries should be immutable. Corrections should create new transactions rather than silently modifying financial history.

33. Testing Strategy

Unit tests: ContributionService · LoanService · LedgerService · PaymentService · ReconciliationService

Integration tests: FastAPI ↔ PostgreSQL

Provider tests: Use provider sandboxes (Jenga, KCB BUNI, Daraja)

Webhook tests: SUCCESS · FAILED · PENDING · CANCELLED · DUPLICATE · UNKNOWN

Financial tests: 1000 contribution · duplicate callback · reversal · partial payment · multiple contributions · loan + repayment

34. Deployment

Initial: FastAPI → Linux VPS / Cloud → PostgreSQL

Production components can later include: Reverse Proxy · Application Server · PostgreSQL · Redis · Background Worker · Monitoring · Logging · Backup System. Do not introduce these unnecessarily in V1.

35. Development Phases

Phase 1 — Requirements · Domain modelling · Database design

Phase 2 — FastAPI · PostgreSQL · Authentication · Chama CRUD

Phase 3 — Members · Roles · Contributions · Shares

Phase 4 — Loans · Repayments · Payouts · Ledger

Phase 5 — Payment model · PaymentProvider interface · Provider resolver

Phase 6 — Jenga integration

Phase 7 — Webhooks · Idempotency · Payment reconciliation

Phase 8 — KCB BUNI · NCBA

Phase 9 — Daraja · Additional providers

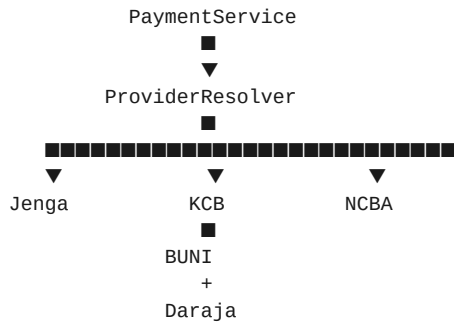
Phase 10 — Bank reconciliation · Statements · Reports

Phase 11 — React · React Native · USSD

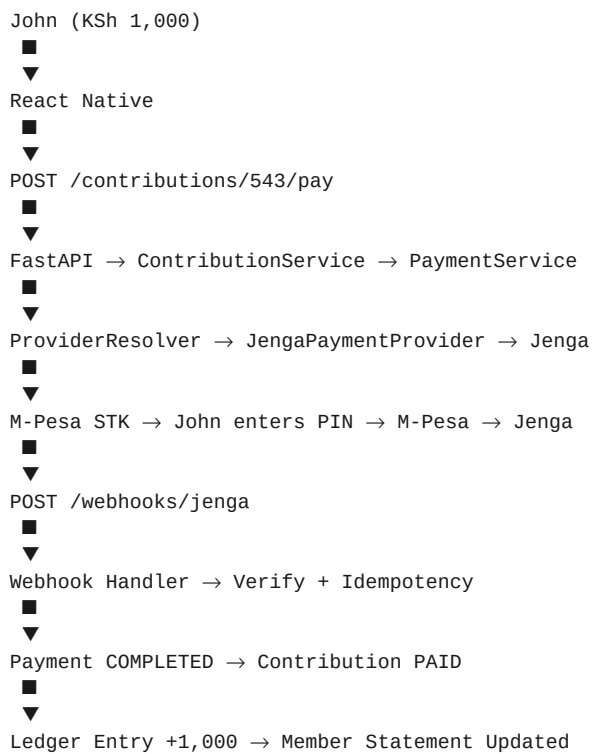
Phase 12 — Security hardening · Monitoring · Backups · Production deployment

36. Provider Abstraction Goal

The ultimate goal is that `payment_service.collect(...)` does not care whether the Chama uses Jenga, KCB BUNI, NCBA or Daraja. The provider layer handles that complexity.



37. Example Complete Payment Flow



38. What We Are NOT Building Initially

To avoid overengineering, V1 will not start with: Microservices · Kafka · gRPC · Kubernetes · Complex event sourcing · Multiple databases · Artificially distributed services.

The initial backend is FastAPI + PostgreSQL + Modular architecture. External providers are integrated behind clean interfaces.

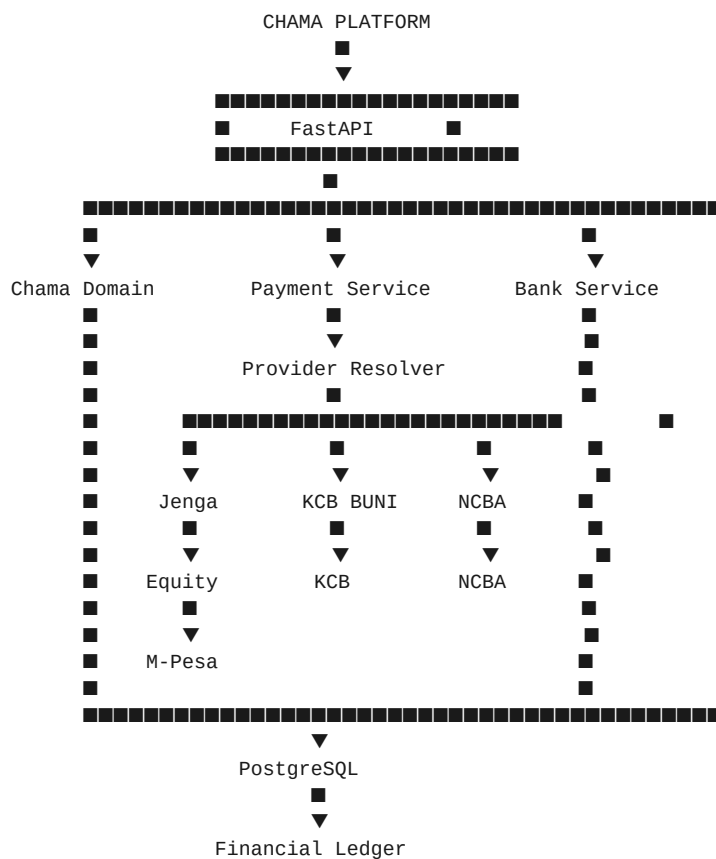
39. Definition of Success

The first meaningful end-to-end milestone is not “we have a dashboard.” It is:

```
Chama created
↓
Member registered
↓
Contribution created
↓
Member clicks Pay
↓
M-Pesa STK
↓
Member enters PIN
↓
Provider confirms payment
↓
Webhook received
↓
Payment reconciled
↓
Contribution marked paid
↓
Ledger updated
↓
Member statement updated
```

Once that works reliably, the platform has a real financial transaction pipeline.

40. Final Architecture



41. Project Philosophy

- 1. Domain first** — Understand the Chama's actual financial processes before writing code.
- 2. Financial correctness** — Money movement must be traceable and auditable.

3. Provider independence — Banks and payment providers are integrations, not the core business logic.

4. Incremental development — Build one working version at a time.

5. Production thinking without premature complexity — Use proper architecture from the beginning, but do not introduce infrastructure that the current scale does not need.

Current Target

The immediate implementation target is V1 (Chama · Members · Roles · Contributions · Shares) then V2 (Loans · Repayments · Payout · Ledger) then V3 (PaymentProvider abstraction) then V4 (Jenga + M-Pesa STK) and so on.

The first coding task should therefore be V1 domain modelling and database design, not the Jenga integration. The payment layer should be designed before the first provider is implemented, so Jenga becomes the first adapter rather than the foundation of the entire system.